

Alphas on orderbook

W WSL | 7月5日修改

基础尝试

Some hints

订单簿外延一直一个方向imb，但价格却总能窜出抬升，（都市价单买入，果断不limit寻求机会，limit一直压着，是散户都在等待时机）

价格试探到一定位置，观察limit是否反应迅速，向下试探底imbalance有无回应，向上试探顶，imbalance有无回应，回应速度。

1. 刀磨时段，暴涨突破时段

2. 价格变化剧烈的时段

3. 价格上涨的imb，wavgimb，外imb 价格方向与imb方向同向

4. 一段时间的价格中枢，中枢之外的订单簿，outimb（第4,5档位价格加权订单簿不平衡）

特殊位置的filter

- **alphaorderbook001, , imb**

我现在有tick数据订单簿，我希望在每个5min末尾，回看过去5min的tick，

计算价格>70分位数的imb（（买一量-卖一量）/（买一量+卖一量）），wavgimb（订单簿5档价格加权平均不平衡度）；

计算价格<30分位数的imb，wavgimb（订单簿5档价格加权平均不平衡度）；

针对这5min内的tick：

计算过去1min价格是上涨的tick所在位置的imb，wavgimb（订单簿5档价格加权平均不平衡度）

计算过去1min价格是下跌的tick所在位置的imb, wavgimb (订单簿5档价格加权平均不平衡度)

针对过去5min内的tick

计算 1min_pct >70分位数的 imb, imb_1minpct, wavgimb, wavgimb_1minpct

计算 1min_pct <30分位数的 imb, imb_1minpct, wavgimb, wavgimb_1minpct

注意, 这里的1minpct都是滚动计算, 针对每个tick计算其与1min前的tick的pct

针对过去5min内的tick

计算价格pct与delay_1min imb_pct 的相关系数

Dynamic 订单簿

alphaorderbook002订单簿扫描后反应

计算过去5min, 订单簿中间价扫过区间的价格上新增的量

当前1档位level变化量

当前1档位, 在过去5min在5档订单簿中同价量的变化

回看过去5min, 计算价格上涨时段, 买方5档位, 从1档位滑到5档位时候本level订单的减小量, 前方 (现在小于5档位) 订单的出现量, 卖方从高档位滑到1档位时候: (低档位订单被吃掉量, 本档位订单的减小量)

回看过去5min, 计算价格下跌时段, 卖方5档位, 从1档位滑到5档位时候本level订单的减小量, 前方 (现在小于5档位) 订单的出现量, 买方从高档位滑到1档位时候: (低档位订单被吃掉量, 本档位订单的减小量)

买卖方订单簿的最大量对应的level, 这个level位置对应价格的1min_pct, 这个价格在过去1min对应的level变化量

对于卖方订单簿, 对于当前卖一价格, 回看到过去5min, 卖1价的最高值的时刻, 记为t (如果当前时刻就是卖一价就是回看5min的最高值, 则该因子记为Nan), 计算从t时刻到现在, t时刻卖一价与当前时刻卖一价之间的挂单量的平均值。

对于买方订单簿, 对于当前买一价格, 回看到过去5min, 买1价的最低值的时刻, 记为t (如果当前时刻就是买一价就是回看5min的最低值, 则该因子记为Nan), 计算从t时刻到现在, t时刻买一价与当前时刻买一价之间的挂单量的平均值。

订单簿的实时韧性，反应能力

对于一个订单簿，回看过去5min，定义：

主动成交发生后，
$$\frac{(\text{该价格处后续挂单量的平均值} - \text{该价格处成交后的剩余订单量})}{(\text{该笔主动成交量})}$$

价格上涨时刻：

1. 上涨结束时刻相对1min前，定义：买方相对1min前新增价格处的订单量和1min前存在的5档价格处的订单增加量，为跟随上涨买量 (follow_upbuy_v)
2. 上涨结束时刻相对1min前，定义：卖方相对1min消失价格处的订单量和1min前存在的5档价格处的订单减少量，为被动吃掉量 (passive_upsell_v)
3. 上涨结束时刻相对1min后，定义：上涨结束时刻的五档价格处，1min后相对刚刚上涨结束，卖方订单的增加量为，被动恢复卖量 (recover_upsell_v)

价格下跌时刻：

1. 下跌结束时刻相对1min前，定义：卖方相对1min前新增价格处的订单量和1min前存在的5档价格处（如存在）的订单增加量，为跟随下跌卖量 (follow_downsell_v)
2. 下跌结束时刻相对1min前，定义：买方相对1min前消失的价格处的订单量和1min前存在的5档价格处（如存在）的订单减少量，为被动吃掉量 (passive_downbuy_v)
3. 下跌结束时刻相对1min后，定义，下跌结束时刻的五档价格处，1min后相对刚刚下跌结束，买方订单的增加量为被动恢复买量 (recover_downbuy_v)

上涨卖方韧性为：
$$\text{recover_upsell_v} / \text{passive_upsell_v}$$

下跌买方韧性为: $\text{passive_downbuy_v} / \text{recover_downbuy_v}$

上涨买方势头为: $(\text{follow_upbuy_v} + \text{passive_upsell_v}) / \text{passive_upsell_v}$

下跌卖方势头为: $(\text{follow_downsell_v} + \text{passive_downbuy_v}) / \text{passive_downbuy_v}$

上涨卖转能力为: $\text{recover_upsell_v} / \text{follow_upbuy_v}$

下跌买转能力为: $\text{recover_downbuy_v} / \text{recover_downbuy_v}$

频率差异

回看过去5min, 买方, 卖方所有价格档位的范围, 所有价格档位成交量的变化量, 记为 buy_AllLevel_Dv ,

sell_AllLevel_Dv

某个买方价位, 成交量的增加次数, buy_addcnt , 成交量的减少次数。 buy_deccnt (激进活跃)

某个卖方价位, 成交量的增加次数, sell_addcnt , 成交量的减少次数。 sell_deccnt

某个买方价位, 成交量的振幅(5min内最大值-最小值), buy_addAmp , 成交量的振幅。 buy_decAmp

某个卖方价位, 成交量的振幅(5min内最大值-最小值), sell_addAmp , 成交量的振幅。 sell_decAmp

某个买方价位, 成交量的std, buy_addstd , 成交量的std, buy_decstd

某个卖方价位, 成交量的std, sell_addstd , 成交量的std, sell_decstd

动态形状因子

- 基本形状

对50event新增ADD订单计算: Ask价与量斜率的平均值. Ask斜率的斜率

- 订单簿重心

订单簿重心 (aggressive) 定义为 订单簿的成交量加权平均价格, 成交量加权平均档位。

买/卖方订单簿的重心档位的ret,

买/卖方订单簿的重心价格-1档位价格, 他的ret, mean, std

买/卖方订单簿的重心价格的1min变化量 - 买/卖方订单簿的1档位价格的1min变化量

回看过去5min, 相同1档位价格时刻, 订单簿的重心的slope, 重心的mean

- **订单簿及动态形状拆分**

1. 全量订单簿变化量

2. 挂单的筹码拆分为

- a. 价格优势方: 新增档位委买量+原档位净委买增加量+主买成交额

- b. 价格劣势方: 被动卖出成交额+原档位净委卖减少量

3. 形成指标表现激进程度:

- a. 新增档位委买量 / 被动卖出成交额

- b. (新增档位委买量 + 净主买成交额) / 净被动卖出成交额

- c. (新增档位委买量 + 原档位净委买增加量) / (被动卖出成交额 + 原档位净委卖减少量)

- d. (新增档位委买量 + 原档位净委买增加量 + 净主买成交额) / (净被动卖出成交额 + 原档位净委卖减少量)

- **订单簿(动态)峰值hump**

- dynamicOrderHump.ipynb



dynamicOrderHump_selfdumpData.i

pynb

76.47KB



1. 计算每1分钟内的所有AddOrder size 最大的那个档位距离mid价格有多少个ticksize,
2. 定义加总的AddOrder size最大的档位为峰档位, 求出: 比峰档位更靠近mid的所有AddOrder size占这5min总AddOrder的百分比。
3. 峰档位size占这5min总AddOrder的百分比。

时间序列建模

订单簿 中间价, imb与滞后1min的corr。

动态簿流动性比例

我现在有modified_df, quotes, trades 分别为如下字段

请你帮我撰写下列因子的代码：

统计过去5min成交量qtr, 记 $\text{wave} = \text{qtr} * \text{scale}$

从卖方，买方quotes中选出累计订单簿的 $\text{size} = \text{wave}$ ，的档位，如：买方1-5档的 $\text{size} \leq \text{wave}$ 1-6 档的 $\text{size} > \text{wave}$ ，则取出1-5档位为研究对象。 $\text{max_level_B} = 5$

1. 分，买卖方，分别在研究对象档位内，计算 $\text{corr}(\text{modified_df.size}, \text{modified_df.level})$

2. 获取过去500个event的 $0 \sim \text{max_level}$ 的全量订单簿 () ,

计算 $\text{quotes.diff}()$ 表示各档位相对上一个档位的size变化量。

计算存在量变化价格档位的净增额 wave_deltasize ，计算不存在量变化价格档位的总size = static_allsize

$\text{wave_turnover} = \text{wave_deltasize} / \text{static_allsize}$ (分 B,S)

$\text{wave_all_turnover} = \text{wave_deltasize} / \text{last_quote_allsize}$ (分 B,S)

3. 根据modified_df, 将档位变化的size, 区分为 $\text{delta_trades}, \text{delta_addcxl}$ 统计在 $0 \sim \text{max_level_move}$ (存在量变化的最大价格档位) 的净增额 $\text{wave_deltasize_trades}, \text{wave_deltasize_addcxl}$, 形成下面四个指标：

$\text{wave_turnover_addcxl} = \text{wave_deltasize_addcxl} / \text{static_allsize_addcxl}$ (分 B,S) trades 同理

$\text{wave_all_turnover_trades} = \text{wave_deltasize_trades} / \text{last_quote_allsize_trades}$ (分 B,S) addcxl 同理

4. 计算 $\text{wave_deltasize} / \text{delta_price}$ (存在量变化价格档位的价差)

$\text{wave_deltasize_addcxl} / \text{delta_price}$ (存在量变化价格档位的价差)

$\text{wave_deltasize_trades} / \text{delta_price}$ (存在量变化价格档位的价差)

事件聚集与事件影响

严重失衡事件数目：

- 统计过去5min内：
 - 买方订单簿总量低于卖方订单簿总量的一半的tick数目；卖方订单簿总量低于买方订单簿总量的一半的tick数目
 - 买1量大于买2+买3+买4量的tick数目；卖1量大于卖2+卖3+卖4量的tick数目；

大额成交后的impact

我现在有逐笔trades 和相应最邻近的5档book depth数据， 请你帮我撰写代码，提取出trade amount 高于过去500个tick的trade avg * 1.5 的 trade 条目定义为大主动卖/买trade，并分别针对主动卖/买 trade 条目，利用其后50个tick的条目数据，计算在大买/卖trade发生后 50个tick，

- 市场的imbalance、五档加权imb的avg 及其分位数，
- imbalance，五档加权imb与 1..50序列的 slope，
- 大trade 价格处前2个挡位的增加量/减少量，后2个挡位的增加量/减少量
- 后50个trade tick 的amount的mean， std 指标, 后面第50个tick相对成交价格的return。后面50个tick 价格的std

研究impact对 新增与整体订单簿影响

我现在有quote 形式如上，请你帮我在这个基础上计算如下代码，统计过去500个event， mid_price 向上变动的时刻，后面的2-5个event 新增的档位的量是多少，同时也计算出向下变动的时刻，给出 $(createsize_uptick - createsize_downtick) / (createsize_uptick + createsize_downtick)$ 请你帮我给出合理时间复杂度好的代码在pandas df 上

此外我还有modified_df 如下：

请你帮我计算在过去的500个event中，

- trd_B_react_B = 当主动Buy trade 发生时候，附近的10-40 个event 中 Buy订单簿的 add-cxl的size是多少，
- trd_S_react_S = 当主动Sell trade 发生时候，附近的10-40 个event 中 Sell订单簿的 add-cxl是多少，

3.trd_B_react_S = 当主动Buy trade 发生时候, 附近的10-40 个event 中 Sell订单簿的 add-cxl是多少,

4.trd_S_react_B = 当主动Sell trade 发生时候, 附近的10-40 个event 中 Buy订单簿的 add-cxl是多少,

最终形成因子 $(\text{trd_B_react_B} - \text{trd_S_react_S}) / (\text{trd_B_react_B} + \text{trd_S_react_S})$

$(\text{trd_B_react_S} - \text{trd_S_react_B}) / (\text{trd_B_react_S} + \text{trd_S_react_B})$

$(\text{trd_B_react_B} - \text{trd_B_react_S}) / (\text{trd_B_react_B} + \text{trd_B_react_S})$

$(\text{trd_S_react_S} - \text{trd_S_react_B}) / (\text{trd_S_react_S} + \text{trd_S_react_B})$

！ *间隔

我定义了事件: acttrd_at_opp_level1_allormore (或者new_level1_add)

请你在每个tick到来的时候, 都向前观察1000个event,

分买卖方向, 统计相邻两个上述事件的时间间隔, 分别得到卖买方向平均值与标准差

计算 trd_opp_more_gap_mean_BSimb

计算 trd_opp_more_gap_std_BSimb

分买卖方向, 统计1000event 中 buyside acttrd_allormore / acttrd, new_level1_add / add 得到 pricechg_acttrd_pct_B; pricechg_add_pct_B

构造因子 pricechg_acttrd_pct_BSimb; pricechg_add_pct_BSimb

计算上述事件在过去1000event中, 发生的时间集中度, 基尼/HHI,

按照gap筛选出, 孤立价格变动

价格变化过大事件

1. 定义订单簿中 买1 卖1价格变化超过一个ticksize(0.01)的位置为事件1

订单簿特殊订单事件

2. 定义订单簿中 买一卖一价格上的订单量为5挡订单中的最高值时候记作事件2

3. 定义在买单价格在买1及以上,卖单在卖1及以下处 新增档位挂单, 并且金额较大记作事件3

- 计算事件2发生时: 买1量, 卖1量, $(BS1-AS1)/(BS1+AS1)$, 未来1min ret, 该订单的存续时间 (orderlifetime), 未来1min同方向新增订单量, 对手方新增订单量, 未来1min的成交量/成交金额。以及事件触发记作1否则为0的Dummy

L3event密集事件

- 过去1min内的trade个数，在ap1 trd的个数、size，在ap2 trd的个数、size。过去1min内在买、卖方对面。ADDMOD的个数，size

停顿的量比：

请你帮我读取 逐笔委托成交逐笔委托数

据， /project/intern5/data/LL_update_data/LL_update_data/LL_datamake79_md1_fea_fast 计算如下因子：

allmorelevel1表示只需要发现 $\text{trd_size} \geq \text{as1/bs1}$ 即可判定是吃掉了一档或者多于一档，askOrder,bidOrder谁小谁是主动方向，gap()表示相邻event的时间差，其他的理解正确。但考虑到由于因子在每个tick 都需要计算因子值，因此取消回看1000个event这类操作，改为 如下的 用 $\text{cumsumfromopen_x} - \text{cumsumfromopen_x.ewm}(\text{halflipe}=500, \text{adjust}=\text{False}, \text{ignore_na}=\text{True}).\text{mean}()$ 的方式近似代替过去1000个event x值的和。

$\text{size_trdact_allmorelevel1_B} = \text{cumsumfromopen_size_trdact_allmorelevel1_B}(\text{sell 的部分为nan, 不符合条件的部分为nan}) - \text{cumsumfromopen_size_trdact_allmorelevel1_B.ewm}(\text{halflipe}=500, \text{adjust}=\text{False}, \text{ignore_na}=\text{True}).\text{mean}$

$\text{size_trdact_B/S}, \text{cnt_trdact_allmorelevel1_B/S}, \text{cnt_trdact_B/S}$ 都是同理计算

$\text{size_trdact_allmorelevel1_B_prob} = \text{size_trdact_allmorelevel1_B} / \text{size_trdact_B}$

$\text{size_trdact_allmorelevel1_S_prob} = \text{size_trdact_allmorelevel1_S} / \text{size_trdact_S}$

$\text{size_trdact_allmorelevel1_prob_BSimb} = (\text{size_trdact_allmorelevel1_B_prob} - \text{size_trdact_allmorelevel1_S_prob}) / (\text{size_trdact_allmorelevel1_S_prob} + \text{size_trdact_allmorelevel1_B_prob})$

$\text{cnt_trdact_allmorelevel1_B} / \text{cnt_trdact_B}$

$\text{cnt_trdact_allmorelevel1_S} / \text{cnt_trdact_S}$

同理计算 $\text{cnt_trdact_allmorelevel1_prob_BSimb}$

`add_newlevel1_B_prob = add_newlevel1_B / add_B`

`add_newlevel1_S_prob = add_newlevel1_S / add_S`

同理计算`cnt/size_add_newlevel1_prob_BSimb`

`trdact_B_diff = cumsumfromopen_trdact_B - cumsumfromopen_trdact_B.ewm(halflife=250, adjust=False, ignore_na=True)`

同理计算`trdact_BSimb_diff`

`shrinkGap_trdact_B = cumsumfromopen_timediff(trdact_B).ewm(halflife=50, adjust=False, ignore_na=True) / cumsumfromopen_timediff(trdact_B).ewm(halflife=1000, adjust=False, ignore_na=True)`

跟随行为

跟随撤单，跟随扫板

时序操作

果断与犹豫（挂单撤单）

@/hft/factors/factor1.py 请你帮我借鉴这个因子构造写法，帮我撰写下面两个因子在factor2中 严格按照factor1 仅仅小部分修改 `_one_stock` 部分

1. 猛拉溢价率因子：

定义

`acttrd_B = (fc == "B") & (tbl_df['orderKind'].shift(-1)=='T')`

```
rush_acttrd_B = orderPrice>tbl_df['sp2'] & acttrd_B (如果是卖方在为bp2) 定义为  
ruchpct_acttrd_B = ((orderPrice - tbl_df['sp2']) / tbl_df['sp2']).where(rush_acttrd_B)  
帮我计算 ruchpct_acttrd_prob_BSimb, cntrush_acttrd_prob_BSimb, sizeruns_acttrd_prob_BSimb
```

2.撤单犹豫度

仿照 factor1_bac 中shrinkGap_trdact_B 构造 shrinkGap_cxl1_B 只撤1挡位的 (判断依据是撤单
functionCode = C, 且askOrder bidOrder 哪一个不为0 代表撤哪一边的单, 观察bv1/sv1(根据撤单方
向定) 相比上一条数据是否减少量, 减少量是否正好为size)

构造 shrinkGap_cxl1_BSimb 和 size_cxl1_prob_BSimb cnt_cxl1_prob_BSimb

cxl1 prob 含义是cxl1 占有所有cxl 事件的cum_minus后的比例
_ratio(tbl_df['f_cnt_cxl1_B'], tbl_df['f_cnt_cxl_B'])

请你修改 @/hft/factors/factor2.py

sizeruns_acttrd_prob_BSimb 帮我把这个也加上,

另外请你根据下面的定义, 修改并添加 size_cxl1_prob_BSimb, cnt_cxl1_prob_BSimb,
cnt_cxlall_prob_BSimb, size_cxlall_prob_BSimb, cnt_cxl1all_prob_BSimb, size_cxl1all_prob_BSimb
cxlall_buy = cxl_is_buy & (tbl_df["bp1"] < bp1_before)
cxl1_buy = cxl_is_buy & (bv1_drop > 0) & (tbl_df["bp1"] == bp1_before)
cxl1all_buy = cxl1_buy | cxlall_buy

抢筹意愿

价格关系:

突破

行为共振

L3 操作专题

- 撤单专题

hitter_percent_cxl - hitter_percent_add

orderlifetime_filtered cxl order.

按照市场价格变化幅度filtered 的撤单周期（切割）

市场价格变化幅度，（挂单/撤单频率）密集时段（委托单数目靠前的）的撤单统计

利用盘口imbalance filter撤单

观察撤单序列的自相关。

Cxl impact (撤单之后是否有回应产生market impact来判断撤单是否有效。)

Add-Cxl pct speed.

我现在有L3data，请你帮我撰写代码，分别在买卖双方计算，在从挂单-撤单的时期，中间价变化
ticksize个数/orderlifetime

从挂单-撤单时期，当中存在了多少新挂单，多少新档位，多少在前面的挂单；

多少成交，主动买卖成交。

多少撤单，多少少档位，多少在前面的撤单（后面存在多少撤单）

其他Idea

- 股指期货沿用股票的日内高频还原数据
- 黄金股与黄金期货的日内贴合，可转债的日内贴合

- 高卓：高价股用maker，低价股用taker
- 换terms换结构，换算子
- add/cxl/trade 对tick up/down 影响力是多少？

Qube 专题：

急切需要上线的因子(给到布鲁斯大合集)：

1. Cxl_priority / prob
2. Trd_Cxl
3. percentage
4. Broker_Id
5. Cxl_readd
6. 6_react
7. LotGamble
8. ReactImpact
9. 17 Million
 - a. time/size/op/edit/partly/ while_add
 - b. new/old_split_ob
10. Gap 分布统计专题

L3探求新的L2 不同的信息：

Orderlifetime priority BrokerID, LifeCycle_of_order, prob change

L3订单簿此时的静态推广到动态的指标

L3订单簿不同位置x 与时间指标（动态） 之间的互相作用

L2 主要关注连续反应逻辑

重构K线上的K线因子

RedWall:

请你帮我读取 逐笔委托成交逐笔委托数

据, /project/intern5/data/LL_update_data/LL_update_data/LL_datamake79_md1_fea_fast 计算如下因子:

allmorelevel1表示只需要发现 $\text{trd_size} \geq \text{as1}/\text{bs1}$ 即可判定是吃掉了一档或者多于一档, askOrder,bidOrder谁小谁是主动方向, gap()表示相邻event的时间差, 其他的理解正确。但考虑到由于因子在每个tick 都需要计算因子值, 因此取消回看1000个event这类操作, 改为 如下的 用 $\text{cumsumfromopen_x} - \text{cumsumfromopen_x.ewm}(\text{halflife}=500, \text{adjust}=\text{False}, \text{ignore_na}=\text{True}).\text{mean}()$ 的方式近似代替过去1000个event x值的和。

$\text{size_trdact_allmorelevel1_B} = \text{cumsumfromopen_size_trdact_allmorelevel1_B}(\text{sell 的部分为nan, 不符合条件的部分为nan}) - \text{cumsumfromopen_size_trdact_allmorelevel1_B.ewm}(\text{halflife}=500, \text{adjust}=\text{False}, \text{ignore_na}=\text{True}).\text{mean}$

size_trdact_B/S, cnt_trdact_allmorelevel1_B/S , cnt_trdact_B/S 都是同理计算

$\text{size_trdact_allormorelevel1_B_prob} = \text{size_trdact_allormorelevel1_B} / \text{size_trdact_B}$
 $\text{size_trdact_allormorelevel1_S_prob} = \text{size_trdact_allormorelevel1_S} / \text{size_trdact_S}$
 $\text{size_trdact_allormorelevel1_prob_BSimb} = (\text{size_trdact_allormorelevel1_B_prob} - \text{size_trdact_allormorelevel1_S_prob}) / (\text{size_trdact_allormorelevel1_S_prob} + \text{size_trdact_allormorelevel1_B_prob})$

$\text{cnt_trdact_allormorelevel1_B} / \text{cnt_trdact_B}$
 $\text{cnt_trdact_allormorelevel1_S} / \text{cnt_trdact_S}$
同理计算 $\text{cnt_trdact_allormorelevel1_prob_BSimb}$

$\text{add_newlevel1_B_prob} = \text{add_newlevel1_B} / \text{add_B}$
 $\text{add_newlevel1_S_prob} = \text{add_newlevel1_S} / \text{add_S}$
同理计算 $\text{cnt_size_add_newlevel1_prob_BSimb}$

$\text{trdact_B_diff} = \text{cumsumfromopen_trdact_B} - \text{cumsumfromopen_trdact_B.ewm(half\text{life}=250, \text{adjust}=\text{False}, \text{ignore_na}=\text{True})}$
同理计算 trdact_BSimb_diff

$\text{shrinkGap_trdact_B} = \text{cumsumfromopen_timediff(trdact_B).ewm(half\text{life}=50, \text{adjust}=\text{False}, \text{ignore_na}=\text{True})} / \text{cumsumfromopen_timediff(trdact_B).ewm(half\text{life}=1000, \text{adjust}=\text{False}, \text{ignore_na}=\text{True})}$

```
<!-- cxl_eff_B / event_all - ewm(cxl_eff_B / event_all, 250) -->
```

@/hft/factors/factor1.py 请你帮我借鉴这个因子构造写法，帮我撰写下面两个因子在factor2中 严格按照factor1 仅仅小部分修改 _one_stock 部分

1.猛拉溢价率因子：

定义

```
acttrd_B = (fc == "B") & (tbl_df['orderKind'].shift(-1)=='T')
```

```
rush_acttrd_B = orderPrice>tbl_df['sp2'] & acttrd_B (如果是卖方在为bp2) 定义为
```

```
ruchpct_acttrd_B = ((orderPrice - tbl_df['sp2']) / tbl_df['sp2']).where(rush_acttrd_B)
```

帮我计算 ruchpct_acttrd_prob_BSimb, cntrush_acttrd_prob_BSimb, sizeruns_acttrd_prob_BSimb

2.撤单犹豫度

仿照 factor1_bac 中shrinkGap_trdact_B 构造 shrinkGap_cxl1_B 只撤1挡位的（判断依据是撤单functionCode = C，且askOrder bidOrder 哪一个不为0 代表撤哪一边的单，观察bv1/sv1(根据撤单方向定) 相比上一条数据是否减少量，减少量是否正好为size)

构造 shrinkGap_cxl1_BSimb 和 size_cxl1_prob_BSimb cnt_cxl1_prob_BSimb

cxl1 prob 含义是cxl1 占有所有cxl 事件的cum_minus后的比例

```
_ratio(tbl_df['f_cnt_cxl1_B'], tbl_df['f_cnt_cxl_B'])
```

请你修改 @/hft/factors/factor2.py

sizeruns_acttrd_prob_BSimb 帮我把这个也加上，

另外请你根据下面的定义，修改并添加

定义size_cxl1_prob_BSimb对于在破前高基础上进一步筛选可能明日隔夜更涨的股票有用，其表示存在由于撤单（撤销了全部的level）而造成价格的抬升，这种事件发生频率越高越说明恐慌撤单。


```

cxlall_buy = cxl_is_buy & (tbl_df["bp1"] < bp1_before)
cxl1_buy = cxl_is_buy & (bv1_drop > 0) & (tbl_df["bp1"] == bp1_before)
cxl1all_buy = cxl1_buy | cxlall_buy
cnt_cxl1_prob_BSimb, cnt_cxlall_prob_BSimb, size_cxlall_prob_BSimb, cnt_cxl1all_prob_BSimb,
size_cxl1all_prob_BSimb

```

代码块

```

1  tbl_df['sp1'] = (tbl_df['sp1'] <= 0, tbl_df["bp1"] * 0.7, tbl_df['sp1'])
2  tbl_df['bp1'] = (tbl_df["bp1"] > 5 * tbl_df['sp1'], tbl_df['sp1'] * 1.3, tbl_df['sp1'])
3  tbl_df["bp1"] = (tbl_df["bp1"] <= 0, tbl_df['sp1'], tbl_df['sp1'])
4  tbl_df['orderPrice'] = np.where((tbl_df['orderKind'].astype(str) == "I") & (tbl_df['functionCode'] == 'B'), tbl_df['price_lob
5                                np.where((tbl_df['orderKind'].astype(str) == "I") & (tbl_df['functionCode'] == 'S'), tbl_df['price_lob
6  tbl_df['orderPrice'] = np.where((tbl_df['orderKind'].astype(str) == "U") & (tbl_df['functionCode'] == 'B'), tbl_df['price_lob
7                                np.where((tbl_df['orderKind'].astype(str) == "I") & (tbl_df['functionCode'] == 'S'), tbl_df['price_lob

```

请你参考 分别为我构造如下因子生成代码：在factor3中

3.拉升打压难度系数

acttrd_B 定义为 $acttrd_B = (fc == "B") \& (tbl_df['orderKind'].shift(-1) == 'T')$

acttrd_B_sizeavgtick = acttrd_B 的size 除以 (trd集群最后的price - trd集群开始的price) ; trd集群定义为, acttrd_B 后连续的tbl_df['orderKind'] == 'T' 的条目

acttrd_B_sizeavgtick_prob = cum_ewm(size_acttrd_B_avgtick) / cum_ewm(size_acttrd_B)

updown_tough = acttrd_sizeavgtick_prob_BSimb

4.连续反映羊群系数

acttrd_B 发生后, 后续20个event 的 acttrd_B 个数/size, addlevel1_S 个数/size, acttrd_S 的个数/size, 分别记作 acttrd_B_acttrd_B_react, acttrd_B_addlevel1_S_react, acttrd_B_acttrd_S_react

$$\text{acttrd_B_}\{\text{acttrd_B}\}_\text{react_prob} = \text{cum_ewm}(\text{acttrd_B_}\{\text{acttrd_B}\}_\text{react}) / \text{cum_ewm}(\text{acttrd_B})$$
$$\text{acttrd_BSimb_}\{\text{acttrd_B}\}_\text{react_prob} = \text{imb}(\text{acttrd_B_}\{\text{acttrd_B}\}_\text{react_prob}, \text{acttrd_S_}\{\text{acttrd_S}\}_\text{react_prob})$$

连续反映羊群系数 加上acttrd_B 发生后 后续20个event 的 cxl_alllevel1 的个数

5.控盘单所在区间价格偏向

在当前mid $\pm$ price.rolling(5000).std()* scale 范围内

$$\text{pctcxltrd_B} = \text{cxl_B 条目的价格均值} / \text{trd_B 条目价格均值} - 1.$$
$$\text{pctcxltrd_BSimb} = \text{imb}(\text{cum_ewm}(\text{pctcxltrd_B}), \text{cum_ewm}(\text{pctcxltrd_S}))$$

1min pct 与 1min amount 相关系数

1min pct 与后 1min amount 的相关系数

1min pct 与前 1min amount 的相关系数

1min pct>0 时刻的bar $\text{corr}(\text{price}, \text{amt}); \text{corr}(\text{price}, \text{amt}_1); \text{corr}(\text{price}, \text{amt}_{-1})$

写在factor3中

请你参考@/hft/factors/factor1.py 帮我构造如下高频因子 在factor4:

6.价格另类抬升比例:

cxl_alllevel_B 定义为 cxl 行(f='C')的 bp1 相对上一行发生变化,

计算 $\text{cxl_alllevel_B_downratio} = \text{cum_ewm}(\text{cxl_alllevel_B}) / \text{cumewm}(\text{cxl_alllevel_B}) + \text{cumewm}(\text{acttrd_S})$

$\text{cxl_alllevel_S_upratio} = \text{cum_ewm}(\text{cxl_alllevel_S}) / \text{cumewm}(\text{cxl_alllevel_S}) + \text{cumewm}(\text{acttrd_B})$

$\text{cxl_alllevel_ratio_BSimb} = _imb(\text{cxl_alllevel_B_downratio}, \text{cxl_alllevel_S_upratio})$

注意前缀可以是size / cnt

请你也撰写

7. CxlCxl_prob

cxl_atlevel1_B 定义为 f='C' 如果 bp1 == bp1.shift() 则要求 bv1!=bv1.shift()

统计当前cxl_atlevel1_B 前面20个event中是否出现了 cxl_atlevel1_B 如果有 记录

cnt_conti_cxl_level1_B = 前面的前面20个event中 cxl_atlevel1_B的个数+1, size_conti_cxl_level1_B 记录为 包括自己cxl_atlevel1_B在内的前面20个event中 cxl_atlevel1_B的size

cxl_eff_B 定义为f='C' 且 bidOrder>0 且 bv1~bv5 至少有一个和bv1.shift()~bv5.shift() 不一样; 如果是 askOrder>0 则为av1~av5

计算 $\text{cxt_atlevel1_B_prob1} = \text{cum_ewm}(\text{conti_cxl_level1_B}) / \text{cum_ewm}(\text{cxl_level1_B})$

计算 $\text{cxt_atlevel1_B_prob2} = \text{cum_ewm}(\text{conti_cxl_level1_B}) / \text{cum_ewm}(\text{cxl_eff_B})$

最终计算 $\text{cxt_atlevel1_prob1_BSimb}$, $\text{cxt_atlevel1_prob2_BSimb}$

请你把@/hft/factors/factor3.py 以及你撰写的factor4 以及@/hft/factors/fac_doc.md 主动学习, 总结到@/.clinerules/hft_tickbytick_factors.md

就现在的问题就还是 都只在涨停票上看对吧(lob只有涨停)

and 预期价格的反映 是否就是: 定义 add_out_S 为挂单在 $\text{mid} + \text{prx_std} * \text{scale}$ 以外的价格的挂单, 用远盘add单的wavg均价 与当前盘口距离反映吗

8. Deepzone:

将卖方价格区间分为

$(, \text{sp1}]$, $(\text{mid}, \text{mid} + \text{prx_std} * 0.5]$, $(\text{mid} + \text{prx_std} * 0.5, \text{mid} + \text{prx_std} * 1]$, ... $(\text{mid} + \text{prx_std} * (\text{l_scale} - 1), \text{mid} + \text{prx_std} * \text{l_scale}]$ 如 $\text{l_scale} = 5$

$\text{cnt}/\text{size_add_}\{bi\}_{bj}_S$ 就是 $(bi, bj]$ 区间的挂单/挂单量, 否则为nan,

$\text{add_}\{bi\}_{bj}_prob1_BSimb = _imb(\text{cum_ewm}(\text{add_}\{bi\}_{bj}_B) / \text{cum_ewm}(\text{add_}0_5_B), \text{cum_ewm}(\text{add_}\{bi\}_{bj}_S) / \text{cum_ewm}(\text{add_}0_5_S))$

$\text{add_}\{bi\}_{bj}_prob2_BSimb = _imb(\text{cum_ewm}(\text{add_}\{bi\}_{bj}_B) / \text{cum_ewm}(\text{add_}B), \text{cum_ewm}(\text{add_}\{bi\}_{bj}_S) / \text{cum_ewm}(\text{add_}S))$

$\text{add_}\{bi\}_{bj}_prob3_BSimb = _imb(\text{cum_ewm}(\text{add_}\{bi\}_{bj}_B) / (\text{cum_ewm}(\text{add_}0_5_B) + \text{cum_ewm}(\text{cxl_}0_5_B)), \text{cum_ewm}(\text{add_}\{bi\}_{bj}_S) / (\text{cum_ewm}(\text{dd_}0_5_S) + \text{cum_ewm}(\text{cxl_}0_5_S)))$

$\text{add_}\{bi\}_{bj}_BSimb = _imb(\text{cum_ewm}(\text{add_}\{bi\}_{bj}_B), \text{cum_ewm}(\text{add_}\{bi\}_{bj}_S))$

同理请你对cxl 也构造

$\text{cnt}/\text{size_cxl_}\{bi\}_{bj}_BSimb$

cnt/size_cxl_{bi}_{bj}_prob1_BSimb

cnt/size_cxl_{bi}_{bj}_prob2_BSimb

cnt/size_cxl_{bi}_{bj}_prob3_BSimb

给出因子构造代码factor4

给出因子构造代码factor6

9. DISTANCE

tbl_df['TIME_BIN'] = np.ceil(tbl_df.time_sec / 300) * 300

分卖买方, groupby TIME_BIN, 求过去5min add_order 的 wavg_price

定义

dis_out_BSimb = _imb(abs(wavg_price_B - mid), abs(wavg_price_S - mid))

该因子每5min一个值 (注意这个因子的写法与原来的_one_stock 写法有不同, 原来是每个tick一个值, 如今需要先分 B/S 把对方事件赋值为nan 然后groupby 每5min 统计值)

10. Deepzone_up_down

观察add/cxl事件发生的前20个tick acttrd_B 的数量与 acttrd_S 的数量谁多,

如果 acttrd_B 多 则 add_{bi}_{bj}_B_up = add_{bi}_{bj}_B

如果 acttrd_S 多 则 add_{bi}_{bj}_B_down = add_{bi}_{bj}_S

否则 add_{bi}_{bj}_B_peace = add_{bi}_{bj}_S

构造

cnt/size_cxl/add_{bi}_{bj}_BSimb_up/down/peace

cnt/size_cxl/add_{bi}_{bj}_prob1_BSimb_up/down/peace

cnt/size_cxl/add_{bi}_{bj}_prob2_BSimb_up/down/peace

cnt/size_cxl/add_{bi}_{bj}_prob3_BSimb_up/down/peace

请你参考 @factor帮我构造如下代码：

11. Cxl_accelarate

请你分别在买卖订单簿上找到在第一挡位撤单的事件 cxl_atlevel1_B, 请你构造因子

```
cxl_atlevel1_acc_BSimb = _imb(_ratio(cum_ewm(cxl_atlevel1_B, short=4000),  
cum_ewm(cxl_atlevel1_B, long=40000)), _ratio(cum_ewm(cxl_atlevel1_S, short=4000),  
cum_ewm(cxl_atlevel1_S, long=40000)))
```

请你同理帮我构造 cxl_atlevel2_acc_BSimb; cxl_atlevel12_acc_BSimb 请你注意参考@factor4 中对于 atlevel1 的判断, atlevel2 判断是：

1. bp2==bp2.shift() ; bv2!=bv2.shift() 说明撤单减少了第二挡位的原有量
2. bp1==bp1.shift(); bp2!=bp2.shift() 说明撤单让第二挡位消失了

12. new_order_wavg_dis

请你计算

time 应当为秒且下午的时间减去了中午的1.5*3600s

```
wavgtime_pricedis_add_lessprxstd_B = cum_ewm(price_add_level15_B * time_add_level15_B) /  
cum_ewm(time_add_level15_B) / mid
```

```
wavgtime_pricedis_add_lessprxstd_S = cum_ewm(price_add_level15_B * time_add_level15_B) /  
cum_ewm(time_add_level15_B) / mid
```

add_lessprxstd 鉴别方式为: side='B' & type=='A' & type.shift(-1)!='T' & price> mid -
prxstd(rolling_window=5000) * 1.2

最终形成 wavgtime_pricedis_add_lessprxstd_BSimb

仿照12 计算如下因子:

12.1 newact_order_wavg_dis

请你计算

time 应当为秒且下午的时间减去了中午的 $1.5 \times 3600s$

$$\text{wavgtime_pricedis_addact_lessprxstd_B} = \frac{\text{cum_ewm}(\text{price_addact_lessprxstd_B} * \text{time_addact_lessprxstd_B})}{\text{cum_ewm}(\text{time_addact_lessprxstd_B})} / \text{mid}$$
$$\text{wavgtime_pricedis_addact_lessprxstd_S} = \frac{\text{cum_ewm}(\text{price_addact_lessprxstd_B} * \text{time_addact_lessprxstd_B})}{\text{cum_ewm}(\text{time_addact_lessprxstd_B})} / \text{mid}$$

addact_lessprxstd 鉴别方式为: $\text{side}='B' \ \& \ \text{type}=='A' \ \& \ \text{price} > \text{mid} - \text{prxstd}(\text{rolling_window}=5000)$ * 1.2

最终形成 wavgtime_pricedis_addact_lessprxstd_BSimb

12.2 cxl_order_wavg_dis

请你计算

time 应当为秒且下午的时间减去了中午的 $1.5 \times 3600s$

$$\text{wavgtime_pricedis_cxl_eff_B} = \frac{\text{cum_ewm}(\text{price_cxl_eff_B} * \text{time_cxl_eff_B})}{\text{cum_ewm}(\text{time_cxl_eff_B})} / \text{mid}$$
$$\text{wavgtime_pricedis_cxl_eff_S} = \frac{\text{cum_ewm}(\text{price_cxl_eff_B} * \text{time_cxl_eff_B})}{\text{cum_ewm}(\text{time_cxl_eff_B})} / \text{mid}$$

cxl_eff_B 鉴别方式为: 定义为 $f='C'$ 且 $\text{bidOrder} > 0$ 且 $\text{bv1} \sim \text{bv5}$ 至少有一个和 $\text{bv1.shift}() \sim \text{bv5.shift}()$ 不一样; 如果是 $\text{askOrder} > 0$ 则为 $\text{av1} \sim \text{av5}$ (cxl_eff_S)

最终形成 wavgtime_pricedis_cxl_eff_BSimb

13. 反映系数:

pct_acttrd_B 表示 acttrd_B 事件发生后的30个event 中, mid价格提升的幅度 即 $\text{mid}_{50} / \text{mid}_1 - 1$. 最终结果显示在当前acttrd_B 30 个event后的那一行

$$\text{pct_acttrd_BSimb} = _imb(\text{cum_ewm}(\text{pct_acttrd_B}), \text{cum_ewm}(\text{pct_acttrd_S}))$$

@/jk/hft/factors/factor7.py 请你帮我修改factor7 定义一个新的BPnx1,SPnx1 是 price_nocross 对于 f='B/S' type.shift(-1)=='T'的条目, BPnx1, SPnx1 分别为其后最后一个连续T 的 bp1,sp1, 如果后面只有一个 T 则就为这个row 的bp1,sp1, 对于其他条目则就取 BP01, SP01

请你参考@factor1

帮我在factor8 中构造如下集中度因子

14. 集中度

现有订单簿上订单额 (过去挂单/过去acttrd/过去撤单 size/gap/time/price) 的一个集中度, 集中部分的价格偏移; 集中部分与现在的距离

定义有效被动挂单

add_eff_B 鉴别方式为: side='B' & type=='A' & type.shift(-1)!='T' & price> mid - prxstd(rolling_window=5000) * 1.0

size_add_eff_B_HHI = _ratio(cum_ewm(size_add_eff_B**2), cum_ewm(size_add_eff_B) ** 2)

size_add_eff_BSimb_HHI = _imb(size_add_eff_B_HHI, size_add_eff_S_HHI)

同理给出 size_add_atlevel1_BSimb_HHI

add_atlevel1_B 定义为: orderKind='B' & type.shift(-1)!='T' & price>=bp1

add_atlevel1_S 定义为: orderKind='S' & type.shift(-1)!='T' & price<=ap1

请你同理给出 size_acttrd_BSimb_HHI; size_cxl_atlevel1_BSimb_HHI; size_cxl_atlevel12_BSimb_HHI; size_cxl_eff_BSimb_HHI

cxl_atlevel1 判断是 f='C' 且

1. $bp1 == bp1.shift()$; $bp2 != bp2.shift()$ 或

2. $bp1 != bp1.shift()$

$cxl_atlevel2$ 判断是 $f='C'$ 且

1. $bp2 == bp2.shift()$; $bv2 != bv2.shift()$ 说明撤单减少了第二挡位的原有量 或

2. $bp1 == bp1.shift()$; $bp2 != bp2.shift()$ 说明撤单让第二挡位消失了

cxl_eff_B 定义为 $f='C'$ 且 $bidOrder > 0$ 且 $bv1 \sim bv5$ 至少有一个和 $bv1.shift() \sim bv5.shift()$ 不一样; 如果是 $askOrder > 0$ 则为 $av1 \sim av5$

定义 $timesec$ 应当为秒且下午的时间应当减去了中午的 $1.5 \times 3600s$

定义 $gap_invpure$ 为当前 row 相对上一条 非 $f='T'$ row 的时间间隔 $+0.01$ 的倒数 $1 / (timesec.diff + 0.01)$

定义 $gap_inv_acttrd_B$ 为当前 $acttrd_B$ 相对 上一条 $acttrd_B$ 事件的时间间隔 $+0.01$ 的倒数

定义 $gap_inv_add_eff_B$ 为当前 add_eff_B 相对 上一条 add_eff_B 事件的时间间隔 $+0.01$ 的倒数

定义 $gap_inv_add_atlevel1_B$ 为当前 $add_atlevel1_B$ 相对 上一条 $add_atlevel1_B$ 事件的时间间隔 $+0.01$ 的倒数

定义 $gap_inv_cxl_eff_B$ 为当前 cxl_eff_B 相对 上一条 cxl_eff_B 事件的时间间隔 $+0.01$ 的倒数

定义 $gap_inv_cxl_atlevel1_B$ 为当前 $cxl_atlevel1_B$ 相对 上一条 $cxl_atlevel1_B$ 事件的时间间隔 $+0.01$ 的倒数

请你同理给出:

$\{timesec/gap_invpure/gap_inv_{\{i\}}_{\{i\}}\}_{BSimb_HHI}$

$i = \{$

add_eff

$add_atlevel1$

$acttrd$

cxl_eff

```
    cxl_atlevel1
}
```

15. 过去成交的位置中心

请你参考factor6 每300s 计算过去所有主动成交事件，对买方 记录：

wavg_price_acttrd_B = 按照size加权的平均价位 / mid (最后TIME_BIN的mid)

对卖方记录 wavg_price_acttrd_B = mid(最后TIME_BIN的mid) / 按照size加权的平均价位

最后得到wavg_price_acttrd_BSimb

14. 对倒出货:买卖方成交的订单，平均的单笔成交金额的差

对于f='B' 的主动成交add行 orderKind.shift(-1)=='T' 的行，统计其后存在多少行T, 记录 num_T_opp_B
在当前行

计算avg_T_sell = size/num_T_opp_B; avg_T_buy = size;

对于f='S' 的主动成交add行 orderKind.shift(-1)=='T' 的行，统计其后存在多少行T, 记录 num_T_opp_S
在当前行

计算avg_T_sell = size; avg_T_buy = size/num_T_opp_S;

计算 avgtrdsize_B = cum_ewm(avg_T_buy); avgtrdsize_S = cum_ewm(avg_T_sell)

f_num_T_opp_B = cum_ewm(num_T_opp_B); f_num_T_opp_S = cum_ewm(num_T_opp_S); // 衡量被动单方面多不多

f_avgnum_T_opp_B = _ratio(f_num_T_opp_B, cum_ewm(cnt_acttrd_B)); f_avgnum_T_opp_S =
_ratio(f_num_T_opp_S, cum_ewm(cnt_acttrd_S)); // 衡量主动打掉的多不多

最终形成因子

avgtrdsize_BSimb; num_T_opp_BSimb; avgnum_T_opp_BSimb

16. 回应主动卖方trd的买方add的有多少cnt avg_size?

请你帮我构造因子, 统计acttrd_B 后的50个event 有多少是 add_atlevel1_S 或者 add_acttrd_S, 统计他们的个数和size 记录为 react_trdadd_S;

给出 react_trdadd_BSimb